
Offline Macro-Aware Reinforcement Learning Pipeline

for the AthenAI Finance-RL Challenge

Adam Maltoni

Quantitative Machine Learning Portfolio Report

adam.maltoni@gmail.com

October 1, 2026

Abstract

This report documents my implementation for the AthenAI Finance-RL competition: an offline portfolio-learning pipeline built around thousands of proprietary trading algorithms, a benchmark trade trace, behavioral clustering, internal market-state features, and macro proxy layers. The central engineering decision was to avoid treating the 14,761 algorithm files as a flat list of assets. Instead, I first reconstructed a daily return panel, filtered unusable series, described each algorithm through a static “personality” vector, reduced the universe into behaviorally coherent super-assets, and then designed an enriched state representation for reinforcement learning. The final reinforcement-learning layer was developed as a DQN/backtesting interface rather than as a completed PPO production result; therefore, the empirical section reports only results that are actually supported by the repository and the regenerated figures. The resulting artifact is both a technical paper and a portfolio piece: it explains the mathematical choices, the trade encoding, the windowing logic, the clustering system, the macro proxies, the observed benchmark behavior, the current limitations, and the next development steps.

Keywords: reinforcement learning, portfolio management, offline trading systems, clustering, factor models, macro proxies, risk-off detection, AthenAI.

1 Challenge Definition and Design Goal

The competition problem can be stated as an offline sequential portfolio-management problem. At each decision date, the system observes a very high-dimensional set of trading algorithms and must decide how capital should be allocated without relying on live external APIs at deployment time. This is close to the reinforcement-learning view of finance: an agent interacts with a market environment, observes state, chooses actions, and receives delayed risk-adjusted rewards [Sutton and Barto, 2018, GeeksforGeeks, 2026, Hambly et al., 2023]. In financial practice the difficulty is not only the RL algorithm. It is the construction of a state that is compact enough to learn from, honest enough not to leak the future, and rich enough to reflect market stress.

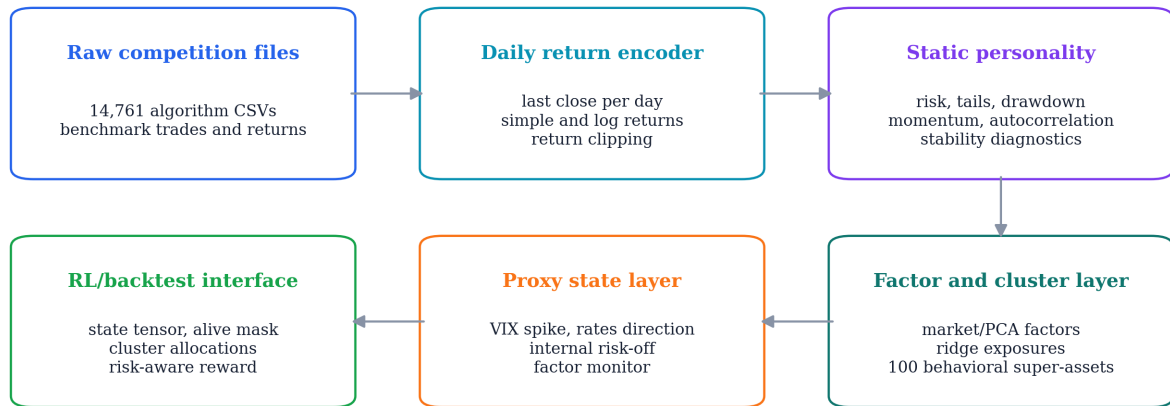
The raw competition universe is unusually wide: thousands of algorithm time series and a benchmark file of executed trades. A naive model that directly learns over all algorithms would be fragile for three reasons. First, many series have short histories, gaps, or nearly constant prices. Second, the market is non-stationary, so static performance metrics alone are not enough. Third, the final decision layer must operate offline, while macro variables such as VIX, rates, or factor returns can be used for training labels but not assumed to be available live.

The formal framing follows standard reinforcement-learning notation and finance-RL surveys, while the practical trading perspective was cross-checked against the course slides by Alberto Suarez and practitioner resources on RL trading, permutation testing, and finance-oriented RL implementations [Suarez, 2025, CodeTrading, 2025, neurotrader, 2025, PyData, 2025, zhonghong9998, 2024].

The implemented solution therefore follows a layered design:

1. reconstruct a reliable daily panel from intraday algorithm CSV files;
2. encode returns and trade traces in a reproducible format;
3. compute static personality features per algorithm;
4. learn clusters of similar algorithms, producing a lower-dimensional set of super-assets;
5. compute rolling cross-sectional, volatility, correlation, drawdown, and factor features;
6. train proxy layers for macro concepts using external data only during training;
7. expose a compact state tensor to an RL/backtesting interface.

This structure follows the spirit of modern RL-for-finance systems [Jiang et al., 2017, Liu et al., 2020, Pippas et al., 2025], but it is tailored to the specific competition constraint: the real asset universe is not a list of public equities, it is a large set of strategy-like algorithm curves.



External macro data are used only to train proxy labels; deployment consumes internal market state and learned proxy predictions.

Figure 1: Implemented pipeline. The external macro information is used to train proxy labels; the deployed state is reconstructed from internal algorithm and cluster behavior.

2 Data Reconstruction

2.1 Raw Algorithm Panel

The repository contains one CSV per algorithm under `data/algoritmos`. Each file stores intraday OHLC rows with a timestamp and a close price. The first preprocessing choice is to collapse every algorithm to a daily close, because the benchmark and most later features are daily or monthly. For algorithm i and calendar day t ,

$$P_{i,t} = \text{last}\{\text{close}_{i,\tau} : \tau \in t\}. \quad (1)$$

From this reconstructed close panel I compute simple and logarithmic daily returns,

$$r_{i,t} = \text{clip} \left(\frac{P_{i,t}}{P_{i,t-1}} - 1, -0.50, 0.50 \right), \quad \ell_{i,t} = \text{clip} (\log P_{i,t} - \log P_{i,t-1}, -0.50, 0.50). \quad (2)$$

The clipping threshold is deliberately wide. It does not suppress ordinary strategy variation; it prevents broken close jumps from dominating rolling statistics, PCA, and distance-based clustering.

Running the regenerated analysis over the repository reads 14,761 algorithm files and produces 8,879,120 daily algorithm rows. Of the 13,663 algorithm identifiers observed with usable rows, 9,491 pass the quality gate. The accepted set covers roughly 69.5% of observed algorithms, with a median of 639 daily observations and a median calendar coverage of 84.6%.

2.2 Quality Gate

The quality gate is simple by design. An algorithm is accepted when it satisfies

$$i \in \mathcal{G} \iff n_i \geq 60, \quad \frac{n_i}{1 + \max(t_i) - \min(t_i)} \geq 0.70, \quad u_i > 2, \quad \hat{\sigma}(r_i) > 10^{-9}, \quad (3)$$

where n_i is the number of daily observations, u_i is the number of unique close values, and the denominator in the coverage ratio is the calendar span in days. I intentionally avoided a performance-based filter at this stage. Filtering by Sharpe or return would leak selection bias into later results; the gate only removes algorithms that are too short, too sparse, or effectively constant.

Universe reconstruction and quality filtering

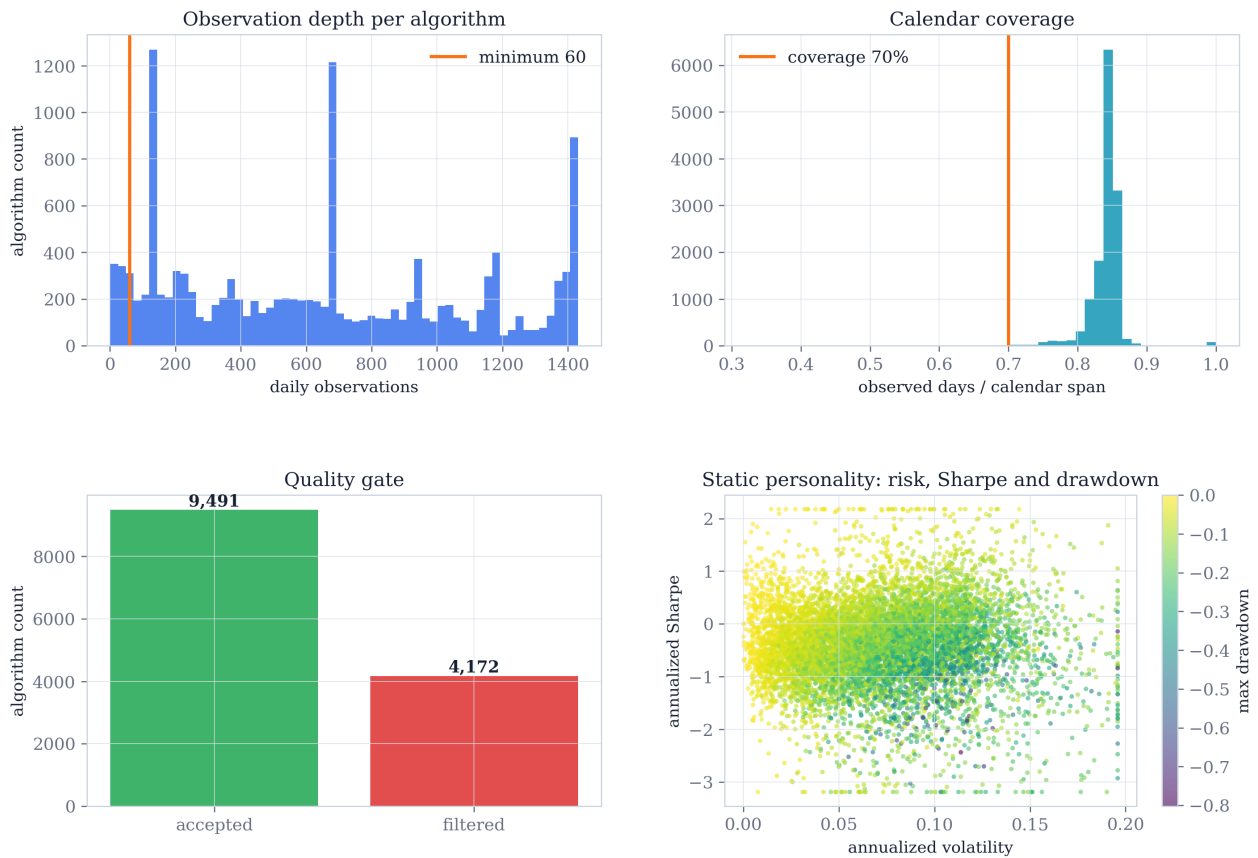


Figure 2: Universe reconstruction and quality filtering. The accepted algorithms are selected by data reliability, not by ex-post performance.

2.3 Return Distribution

The accepted universe still has a strong spike around zero and fat empirical tails. This is consistent with strategy-like curves: many algorithms are inactive or very stable on a given day, while a smaller fraction experiences discrete moves. The reconstructed return distribution contains 6,974,986 accepted daily return observations. The empirical mean is close to zero at -0.008% per day and the standard deviation is 0.493%. The 1% and 99% quantiles are -1.67% and 1.53%, respectively.

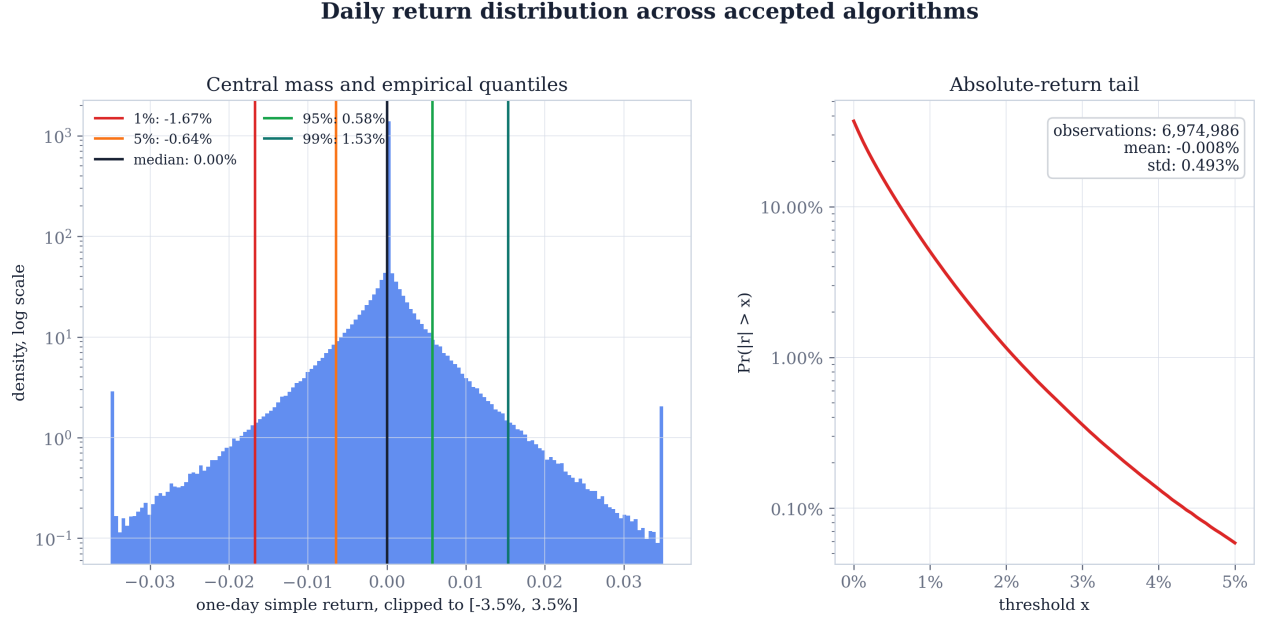


Figure 3: Distribution of accepted algorithm daily returns. The left panel uses a log-density scale so the central mass and tails remain visible; the right panel reports the empirical absolute-return tail.

3 Benchmark Trade Encoding

The benchmark file is more than a performance reference. It is also a trace of how the organizer’s strategy moved through the algorithm universe: each row has a product name, a signed volume, open and close dates, invested amount, AUM, and end-of-day equity fields. I parse `dateOpen` and `dateClose`, compute holding periods, and treat `productname` as the algorithm identifier.

For a trade j in algorithm $i(j)$, volume v_j , opening date o_j , and closing date c_j , the natural position trace is

$$h_{i,t} = \sum_{j:i(j)=i} v_j \mathbf{1}\{o_j \leq t < c_j\}. \quad (4)$$

A normalized allocation trace can then be defined as

$$w_{i,t} = \frac{h_{i,t}}{\epsilon + \sum_k |h_{k,t}|}, \quad (5)$$

where ϵ prevents division by zero. This encoding is useful for forensics, imitation-style diagnostics, and sanity checks on the final action space. It is not used as a claim that the benchmark is optimal.

The benchmark file contains 5,394 trades across 271 distinct algorithms, with a median holding period of 26 calendar days. The concentration of openings in 2024 is visible in Figure 4, and it matters: any model comparison must be aware that the benchmark did not use the full algorithm universe uniformly over the full sample.

Benchmark trade file as the supervised action trace

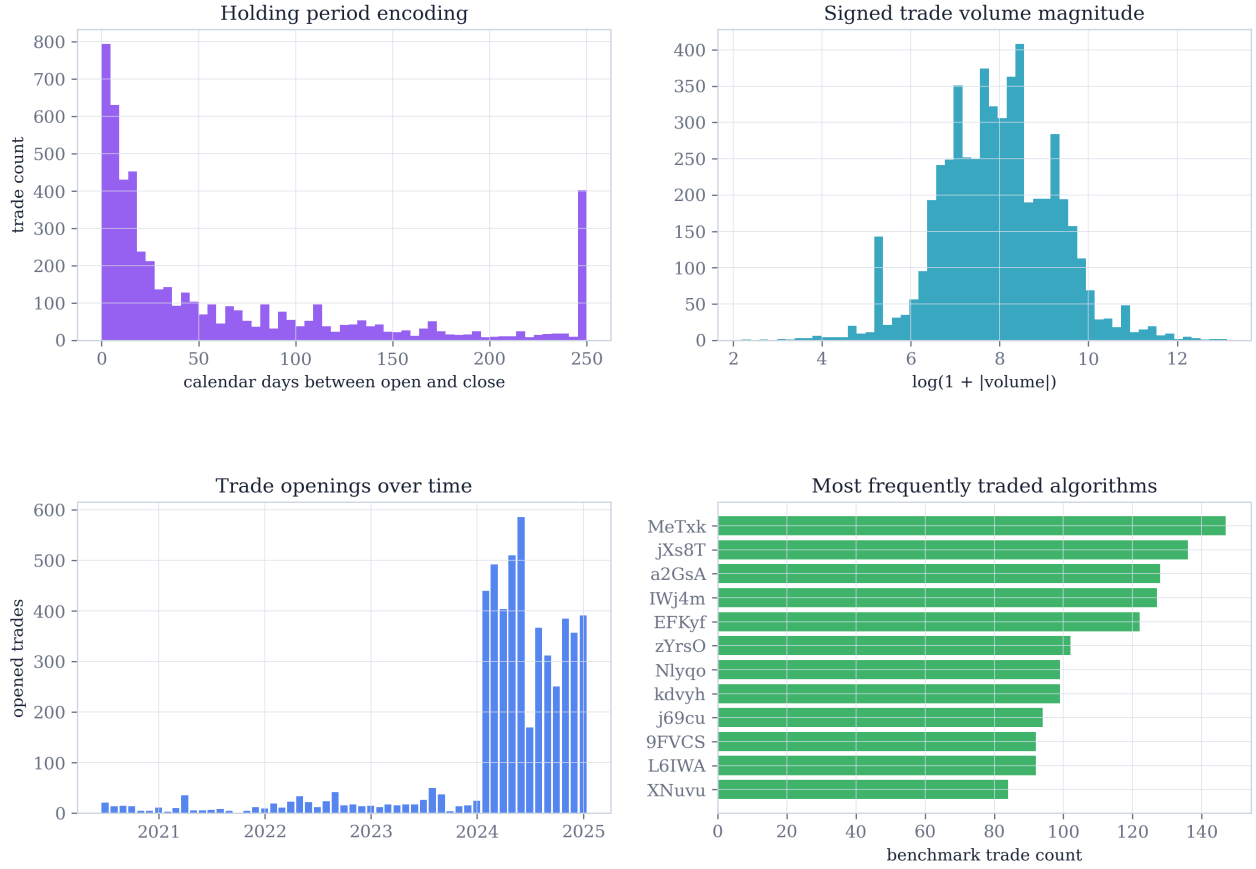


Figure 4: Benchmark trade file diagnostics. The trade file is encoded as an action trace through product identifiers, signed volume, opening dates, closing dates, and holding periods.

4 Static Algorithm Personality

4.1 Why Personality Features?

The main idea behind my solution is that the algorithms should not be treated as anonymous columns. Each curve has a behavioral profile: volatility, tail shape, drawdown depth, tendency to stay underwater, trend, momentum, return autocorrelation, and stability across time. These features are what I call the algorithm's static personality. They give the clustering layer economic meaning, and they make the action space more stable than directly selecting from thousands of raw algorithms.

For each accepted algorithm i , the implemented personality vector ϕ_i includes:

- return location and dispersion: mean, standard deviation, annualized volatility;
- risk-adjusted performance: Sharpe and Sortino ratios;
- tail behavior: empirical quantiles, tail ratios, skewness, excess kurtosis;
- drawdown structure: maximum drawdown, ulcer index, average drawdown depth, time in draw-down;
- temporal behavior: one-day return autocorrelation, absolute-return autocorrelation, 120-day log momentum;
- stability diagnostics: first-half and second-half Sharpe/volatility drift in the full implementation.

The core formulas are standard but important. With $\bar{r}_i$ and $\hat{\sigma}_i$ estimated from the accepted daily returns,

$$\text{Sharpe}_i = \sqrt{252} \frac{\bar{r}_i}{\hat{\sigma}_i}, \quad \text{Sortino}_i = \sqrt{252} \frac{\bar{r}_i}{\hat{\sigma}_{i,-}}, \quad (6)$$

where $\hat{\sigma}_{i,-}$ is the downside deviation. Drawdown is computed on the close curve:

$$DD_{i,t} = \frac{P_{i,t}}{\max_{\tau \leq t} P_{i,\tau}} - 1, \quad MDD_i = \min_t DD_{i,t}, \quad UI_i = \sqrt{\frac{1}{T_i} \sum_t DD_{i,t}^2}. \quad (7)$$

The 120-day momentum is

$$m_{i,t}^{120} = \log P_{i,t} - \log P_{i,t-120}. \quad (8)$$

I include autocorrelation of both returns and absolute returns because they capture different effects: directional persistence versus volatility clustering.

5 Factor Model and Clustering

5.1 Market and PCA Factors

The factor layer begins with an internal market factor. On each day t , over the alive algorithm set $\mathcal{A}_t$,

$$f_t^{mkt} = \frac{1}{|\mathcal{A}_t|} \sum_{i \in \mathcal{A}_t} r_{i,t}. \quad (9)$$

The repository also implements PCA factors over a standardized return matrix. Let $Z \in \mathbb{R}^{T \times N}$ be the matrix of standardized returns after date sampling and missing-data handling. A truncated singular value decomposition

$$Z \approx U_K \Sigma_K V_K^\top \quad (10)$$

produces factor time series $F = U_K \Sigma_K$ and algorithm loadings V_K . Those PCA factors are later used both for universe features and for an alternative correlation-embedding clustering family.

Given a factor matrix $X_t = [1, f_t^{mkt}, f_t^{pca1}, \dots, f_t^{pcaK}]$, each algorithm receives ridge factor exposures:

$$\hat{\beta}_i = (X_i^\top X_i + \lambda I)^{-1} X_i^\top r_i, \quad \lambda = 10^{-3}. \quad (11)$$

The exposure vector complements the static personality vector. Two algorithms can have similar Sharpe ratios but very different market betas or PCA loadings; clustering should see that difference.

5.2 Behavioral Clustering

The main clustering configuration is **behavioral_k100_v1**. For each algorithm,

$$x_i = [\phi_i, \hat{\beta}_i, R_i^2, \hat{\sigma}_i^{resid}], \quad (12)$$

where ϕ_i is the static personality vector and the remaining terms are factor exposure diagnostics. Features are robust-scaled,

$$z_{i,j} = \frac{x_{i,j} - \text{median}(x_{\cdot,j})}{\text{IQR}(x_{\cdot,j}) + \epsilon}, \quad (13)$$

and clustered by k-means or mini-batch k-means:

$$\min_{\{c_k\}_{k=1}^K} \sum_i \min_{k \in \{1, \dots, K\}} \|z_i - c_k\|_2^2, \quad K = 100. \quad (14)$$

I also implemented a correlation/PCA embedding cluster set, **corr_pca_k60_v1**. Its feature matrix is built from PCA loadings rather than the behavioral personality vector. The two cluster families

serve different purposes: behavioral clusters are easier to interpret, while correlation embeddings emphasize co-movement. The article figures focus on the behavioral 100-cluster version because it is the core state compression used downstream.

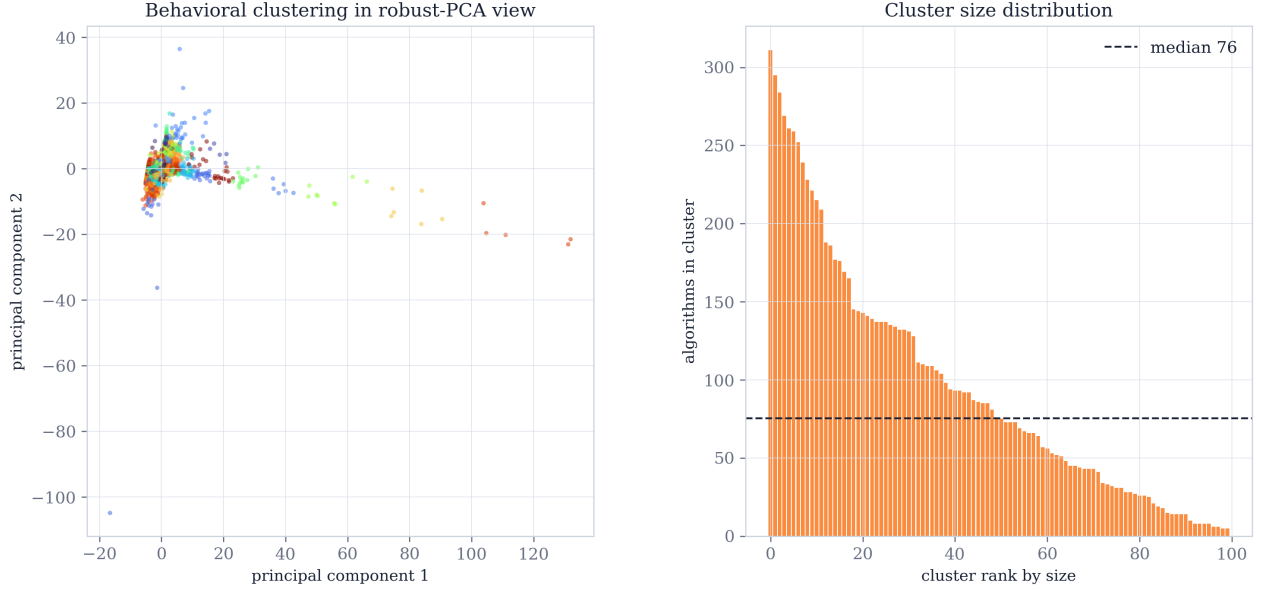


Figure 5: Behavioral clustering of accepted algorithms. The left panel shows a two-dimensional PCA projection of the robust-scaled personality matrix; the right panel shows the 100-cluster size profile.



Figure 6: Cluster-level personality diagnostics. Marker size reflects the number of algorithms in the cluster. The map is intended for model design, not as a final trading signal.

5.3 Cluster Time Series and Alive Masks

Once every algorithm has a cluster label $g(i)$, I build a cluster-level return panel:

$$R_{c,t} = \frac{1}{|\mathcal{A}_{c,t}|} \sum_{i: g(i)=c, i \in \mathcal{A}_t} r_{i,t}. \quad (15)$$

The alive ratio

$$a_{c,t} = \frac{|\mathcal{A}_{c,t}|}{|\mathcal{A}_{c,\cdot}|} \quad (16)$$

is stored because missingness is informative: a cluster with few alive members is a riskier target for allocation. The code also computes rolling cluster features such as 5-, 10-, 20-, and 60-day returns, 20-day annualized volatility, 60-day Sharpe, rolling drawdown proxies, and cluster metadata joined back as static features.

6 Window System and State Features

6.1 Rolling Windows

The repository uses several windows because no single horizon captures the problem:

- 5 and 10 days capture short-term reaction and jump behavior;
- 20 days approximates a trading month and is used for realized volatility;
- 60 days captures a quarterly regime view and is used for Sharpe and drawdown proxies;
- 120 days captures slower momentum in the static personality layer.

This is not arbitrary feature inflation. The windows separate reaction speed. A cluster that looks calm over 60 days but has spiked over 5 days should be represented differently from a cluster that is structurally volatile.

6.2 Cross-Sectional Universe Features

From the cluster return panel, the proxy and RL state layers compute an internal market state

$$u_t = [f_t^{mkt}, \text{breadth}_t, \text{dispersion}_t, \text{tail}_{0.05,t}, \text{tail}_{0.95,t}, \bar{\sigma}_{20,t}, \bar{\rho}_{20,t}, \bar{\rho}_{60,t}, \text{jumpiness}_t, DD_{60,t}^{min}, \dots].$$

For example,

$$\text{breadth}_t = \frac{1}{C_t} \sum_{c=1}^{C_t} \mathbf{1}\{R_{c,t} > 0\}, \quad \text{dispersion}_t = \text{Std}_c(R_{c,t}). \quad (17)$$

The average correlation features are computed from rolling cluster-return correlation matrices. In stress periods, correlations often rise while breadth falls; those two effects are more robust than raw aggregate return alone.



Figure 7: Internal universe state reconstructed from cluster returns. The state includes market return, cross-sectional breadth, volatility, and correlation features.

7 Macro Proxy Layers

7.1 Why Proxies Are Needed

The competition constraint makes live macro dependency dangerous. It is tempting to query VIX, Treasury yields, recession indicators, or Fama-French factors directly, but a deployable offline model should not assume access to those series. My compromise is to use external series only during training to create targets and diagnostic labels, then train internal proxy models that infer those concepts from the algorithm universe itself.

This is the purpose of the `layers` pipeline. It fetches external data for the training period, aligns it with internal universe features, trains small linear/logistic models with walk-forward validation, calibrates probabilities when needed, and writes enriched state columns. In prediction mode, the model consumes internal features and trained proxy models only.

7.2 VIX Spike Proxy

The VIX proxy defines log-level and log-change targets,

$$y_{t,h}^{VIX} = \log(VIX_{t+h}) - \log(VIX_t), \quad (18)$$

and the spike classification target

$$y_{t,h}^{spike} = \mathbf{1} \left\{ y_{t,h}^{VIX} > Q_{0.90}(y_{\cdot,h}^{VIX}) \right\}. \quad (19)$$

The implemented features include internal volatility, market cluster returns, PCA factors, and stress features. The README diagnostics report VIX spike AUC around 0.50 ± 0.10 , which is essentially a baseline. The useful part of this result is negative: the current internal features are not enough to recover VIX spike behavior robustly, even though calibration with Platt scaling reduces ECE from 0.036 to 0.0007.

7.3 Rates Direction Proxy

The rates proxy uses the 10-year Treasury yield change over a horizon of 10 days:

$$\Delta bps_t = 10^4(DGS10_{t+10} - DGS10_t). \quad (20)$$

The target is categorical:

$$y_t^{rates} = \begin{cases} 0, & \Delta bps_t < -5, \\ 1, & |\Delta bps_t| \leq 5, \\ 2, & \Delta bps_t > 5. \end{cases} \quad (21)$$

This is the strongest proxy in the current repository. The documented validation reports sign accuracy of $80.6\% \pm 10.8\%$, and the shuffle leakage test separates real and shuffled targets (0.068 versus 0.028 in the stored diagnostic). This does not make the proxy a trading model by itself; it means the internal universe carries some rates-direction information.

7.4 Internal Risk-Off Target

The risk-off proxy is deliberately internal. Let f_t^{mkt} be the cluster market factor. Over the next $H = 10$ days, compute the worst cumulative forward return:

$$m_t^{(10)} = \min_{1 \leq h \leq 10} \left(\prod_{k=1}^h (1 + f_{t+k}^{mkt}) - 1 \right). \quad (22)$$

The binary target is the bottom decile:

$$y_t^{risk} = \mathbf{1}\{m_t^{(10)} < Q_{0.10}(m^{(10)})\}. \quad (23)$$

Regenerating this target from the current repository gives a threshold of -0.301% and a positive rate of 9.93%. The current model diagnostic in the README reports AUC around 0.46 ± 0.10 , so this layer is not yet predictive. Still, the target itself is valuable because it defines the stress state without importing future external macro labels.

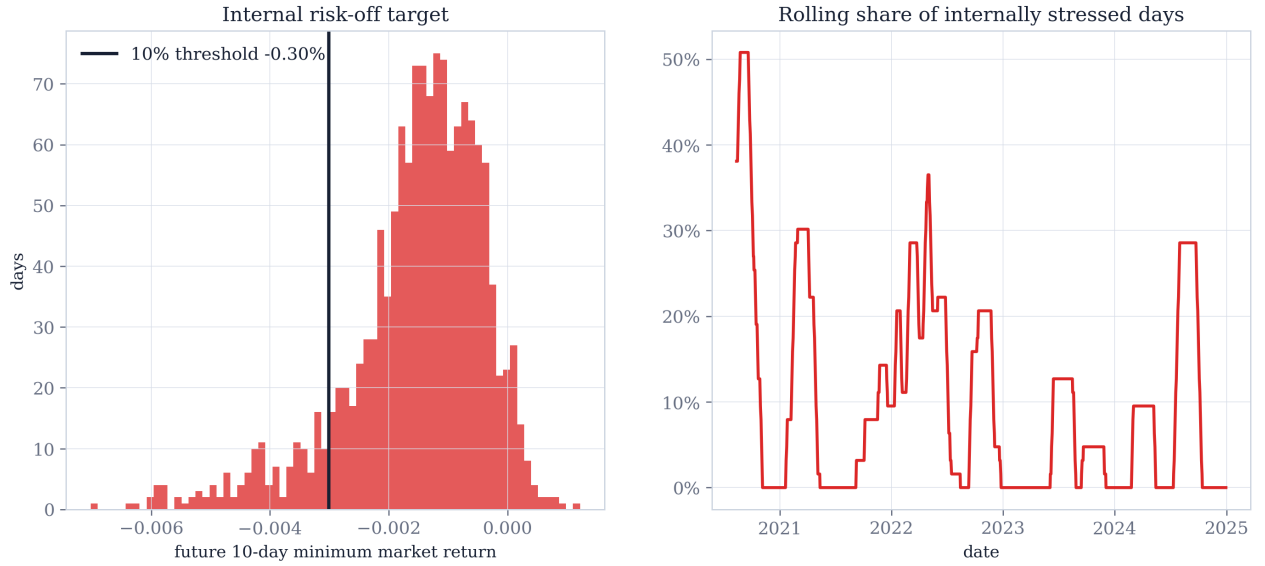


Figure 8: Internal risk-off target generated from the future 10-day minimum return of the cluster market factor. The threshold is the empirical bottom decile.

7.5 Factor Monitor

The factor monitor aggregates universe features to monthly frequency and predicts whether a selected Fama-French factor, especially momentum, is positive in the next month. The current sample contains only 54 monthly observations, so this component is better treated as a diagnostic and future-work module than as a production signal.

Table 1: Current proxy-layer diagnostics documented in the repository. These are not presented as final trading results; they show which macro concepts the internal universe currently can and cannot reconstruct.

Layer	Reported metric	Interpretation
VIX spike	AUC 0.50 ± 0.10 ; ECE $0.036 \rightarrow 0.0007$ after calibration	Classification is baseline-like, but probability calibration works.
Rates direction	Sign accuracy $80.6\% \pm 10.8\%$; leakage test real 0.068 vs shuffled 0.028	Strongest current proxy; internal features appear to encode rates movement.
Risk-off	AUC 0.46 ± 0.10 ; feature leakage check passed with all correlations below 0.9	Target definition is sound, but current predictors are weak.
Factor monitor	54 monthly samples	Useful as a monitoring layer, too small for strong claims.

8 Investment Clock Experiment

In addition to the main cluster pipeline, I implemented an investment-clock experiment in the notebooks. The idea is to classify macro states using inflation and industrial-production growth, then study which algorithms perform in each regime. Let

$$\pi_t = \text{YoY}(CPI_t), \quad g_t = \text{YoY}(INDPRO_t). \quad (24)$$

The thresholds are rolling medians shifted by one month to avoid lookahead:

$$\tilde{\pi}_t = \text{median}(\pi_{t-36:t-1}), \quad \tilde{g}_t = \text{median}(g_{t-36:t-1}). \quad (25)$$

This defines four states:

Regime	Condition	Economic reading
Reflation	$g_t < \tilde{g}_t, \pi_t < \tilde{\pi}_t$	weak growth, low inflation
Recovery	$g_t \geq \tilde{g}_t, \pi_t < \tilde{\pi}_t$	improving growth, contained inflation
Overheat	$g_t \geq \tilde{g}_t, \pi_t \geq \tilde{\pi}_t$	strong growth, high inflation
Stagflation	$g_t < \tilde{g}_t, \pi_t \geq \tilde{\pi}_t$	weak growth, high inflation

The notebook then computes per-algorithm Sharpe by regime and clusters algorithms by the four-dimensional regime-performance profile. This is a second, economically interpretable clustering view. It is not yet the main state in the package pipeline, but it is one of the most promising directions because it links algorithm behavior to macro regimes rather than only to statistical similarity.

9 RL Formulation

9.1 State

After clustering, the action space is reduced from thousands of algorithms to $C = 100$ behavioral clusters. A decision state can be written as

$$s_t = [\mathcal{W}_{t-L+1:t}, u_t, \hat{p}_t^{\text{proxy}}, a_t, w_{t-1}], \quad (26)$$

where $\mathcal{W}_{t-L+1:t}$ is a rolling window of cluster returns and cluster features, u_t is the internal universe state, $\hat{p}_t^{\text{proxy}}$ contains proxy probabilities or scores, a_t is the alive mask, and w_{t-1} is the previous allocation. Including w_{t-1} is important because transaction costs depend on turnover, not only on the next allocation.

9.2 Action

The natural continuous action is a portfolio vector

$$w_t \in \Delta^C = \left\{ w \in \mathbb{R}^C : \sum_{c=1}^C w_c = 1, w_c \geq 0 \right\}. \quad (27)$$

For a DQN implementation, this continuous simplex must be discretized, for example into actions such as holding the previous portfolio, shifting weight toward a selected cluster, moving to cash, or selecting among predefined cluster baskets. The repository currently contains the DQN scaffolding, replay buffer, target network, epsilon-greedy action selection, and backtesting utilities. PPO is referenced in earlier drafts but is not implemented as a trained competition result in the current codebase.

9.3 Reward and Backtest

For cluster return vector R_{t+1} , portfolio return after turnover cost can be written as

$$r_{t+1}^p = w_t^\top R_{t+1} - \kappa \|w_t - w_{t-1}\|_1. \quad (28)$$

The backtest module tracks equity,

$$E_{t+1} = E_t(1 + r_{t+1}^p), \quad (29)$$

and evaluates Sharpe, maximum drawdown, win rate, and Calmar ratio:

$$\text{Calmar} = \frac{\text{annualized return}}{|\text{maximum drawdown}| + \epsilon}. \quad (30)$$

The development direction is to train with a reward that penalizes volatility and drawdown while evaluating with these portfolio metrics, rather than optimizing one noisy annualized score directly.

9.4 DQN Update

The DQN agent follows the usual target-network update [Mnih et al., 2015]. For transition $(s_t, a_t, r_{t+1}, s_{t+1})$, the target is

$$y_t = r_{t+1} + \gamma \max_{a'} Q_{\theta^-}(s_{t+1}, a'), \quad (31)$$

and the loss is

$$\mathcal{L}(\theta) = \mathbb{E} \left[(y_t - Q_{\theta}(s_t, a_t))^2 \right], \quad (32)$$

implemented with a replay buffer, target network, Smooth L1 loss, and epsilon decay. PPO remains a valid future candidate [Schulman et al., 2017], especially if the action is kept continuous over cluster weights, but the current repository does not support reporting PPO performance.

Algorithm 1 Implemented offline state-building workflow

Require: Algorithm CSV files, benchmark trades, yearly/monthly benchmark returns

- 1: Collapse intraday rows to daily closes $P_{i,t}$
 - 2: Compute clipped simple and log returns using Eq. (2)
 - 3: Filter algorithms with Eq. (3)
 - 4: Build static personality vectors ϕ_i with risk, drawdown, tail, momentum, and stability features
 - 5: Estimate internal market and PCA factors; compute ridge exposures $\hat{\beta}_i$
 - 6: Robust-scale $[\phi_i, \hat{\beta}_i]$ and cluster accepted algorithms into super-assets
 - 7: Aggregate cluster returns, alive masks, and rolling window features
 - 8: Build proxy labels for VIX, rates, risk-off, and factors using train-only external data
 - 9: Emit enriched state tensors for the RL/backtesting layer
-

10 Empirical Results

10.1 Benchmark Returns

The benchmark yearly return file shows positive returns in every available year. The best year is 2022 with 7.13%, followed by 2024 with 6.99%. The weakest year is 2020 with 0.54%, noting that the available monthly file begins in June 2020. Compounding the monthly return file gives a total return of 22.95% over the period, with a maximum monthly-resolution drawdown of -1.87%. This drawdown figure should not be over-interpreted as intramonth risk; it is computed from monthly benchmark returns because that is the available benchmark return frequency.

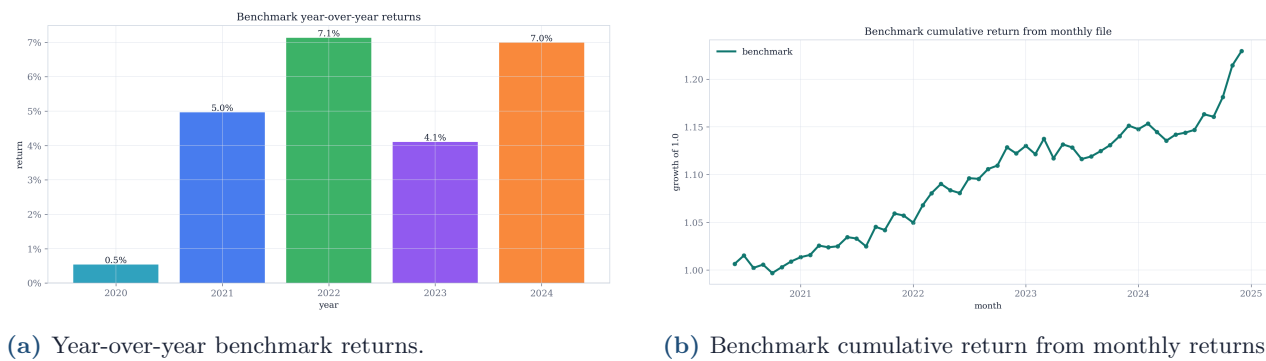


Figure 9: Benchmark performance reconstructed from the repository return files.

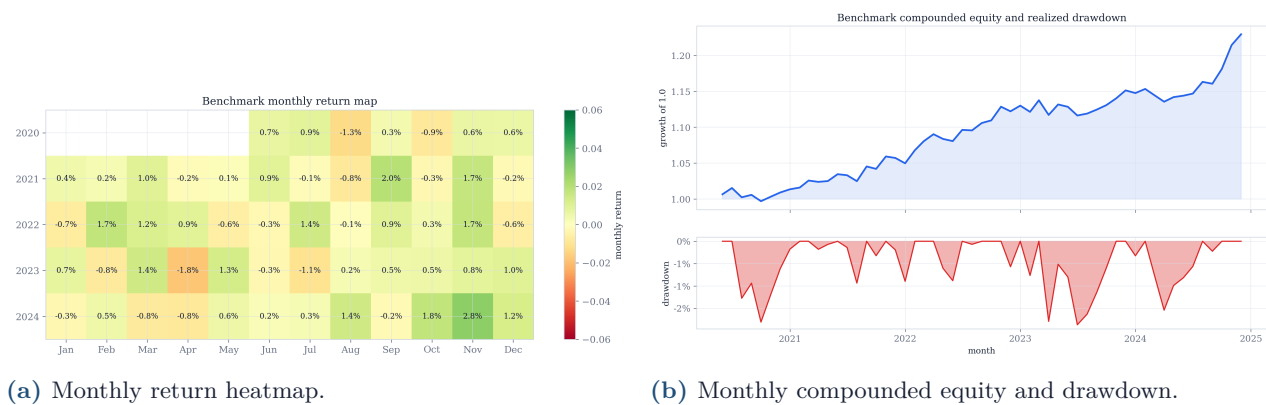


Figure 10: Benchmark monthly structure. The drawdown panel is based on monthly returns, not on fabricated daily model equity.

10.2 What the Results Say About the Model Design

The results are not a final leaderboard claim. They are an audit of the system that has been built:

- the raw universe can be reconstructed into a clean daily panel with millions of observations;
- roughly 9.5k algorithms survive non-performance-based filtering;
- behavioral clustering compresses the action space to 100 super-assets while preserving interpretable risk and return differences;
- the benchmark trade file is concentrated in a small subset of algorithms and a later part of the sample;
- internal breadth, volatility, and correlation features show meaningful time variation;
- rates direction is the strongest macro proxy so far; VIX spike and risk-off prediction require more work.

The most important observation is that the clustering layer is not cosmetic. It changes the learning problem. Instead of asking a neural network to discover structure across thousands of sparse algorithms, the model receives a smaller number of behaviorally coherent time series with metadata, alive masks, and rolling state descriptors. That is the core contribution of this implementation.

11 Implementation Notes

Table 2: Repository components reflected in this report.

Component	Role in the methodology
<code>massive.ipynb</code>	Original preprocessing logic: CSV scan, daily close reconstruction, return clipping, metadata, quality gate, rolling features.
<code>src/athenai/features/personality.py</code>	Static personality feature builder: drawdown, tails, Sharpe/Sortino, autocorrelation, momentum, stability.
<code>src/athenai/factors</code>	Internal market/PCA factors and ridge exposure estimation.
<code>src/athenai/clustering</code>	Behavioral and correlation-embedding clustering, robust scaling, cluster time series, alive masks.
<code>src/athenai/layers</code>	Proxy datasets, walk-forward validation, calibration, leakage tests, enriched state generation.
<code>massive_investment_clock.ipynb</code>	Investment-clock experiment and regime-profile clustering.
<code>src/agents/dqn.py</code>	DQN scaffolding with replay buffer, target network, epsilon-greedy policy, and Smooth L1 update.
<code>src/utils/backtest.py</code>	Portfolio backtesting and performance metrics.

12 Limitations

The main limitation is that the final RL agent is not yet an end-to-end validated competition policy. The repository contains a DQN implementation and a backtest interface, while PPO is not implemented as a trained result. For that reason, I do not report a PPO Sharpe ratio, PPO drawdown, or synthetic learning curve.

There are also data and modeling constraints. The current checkout does not include the generated cache directory, so the figures in this article reconstruct the core analysis directly from raw algorithm and benchmark files. Some package-level pipeline modules reference missing cached artifacts or data helpers; the codebase is best viewed as an active research system rather than a packaged library. The proxy layers show mixed predictive strength: rates are promising, VIX spikes are baseline-like, and risk-off classification is not yet useful as a standalone predictor.

Finally, the benchmark itself is not a clean supervised target for all dates and all algorithms. It trades 271 algorithms out of a much larger universe, with trade openings concentrated in the later part of the sample. That makes the benchmark valuable for forensics and action-trace encoding, but risky as the only source of imitation learning.

13 Future Work

The next version should focus on five concrete improvements:

1. finish the cluster-level RL environment with explicit transaction costs, cash handling, alive-mask constraints, and walk-forward splits;
2. train a continuous-action policy such as PPO or SAC over cluster allocations, while keeping DQN as a discrete-action baseline;
3. strengthen the risk-off proxy with better temporal validation, richer stress features, and calibration diagnostics;
4. integrate the investment-clock regime profile into the main cluster metadata and state tensor;
5. add a full experiment registry so every figure, metric, cache artifact, and model checkpoint is reproducible from a single run id.

The most attractive research direction is the combination of behavioral clustering and regime-aware allocation. If each algorithm is described by both static personality and macro-regime performance, the RL policy can learn not only which clusters look good on average, but which clusters are useful under specific stress, inflation, rates, or momentum conditions.

14 Conclusion

This project is best understood as a serious offline state-construction and portfolio-learning system, not as a thin wrapper around a generic RL tutorial. The key work is the transformation of thousands of raw algorithm curves into a structured decision problem: reliable daily returns, non-leaky quality filtering, personality features, behavioral clusters, internal universe state, proxy macro labels, trade-trace encoding, and an RL/backtesting interface.

The current empirical evidence supports the pipeline design rather than a final agent-performance claim. The benchmark is positive and relatively smooth at monthly resolution; the universe contains enough accepted algorithms to justify clustering; the internal state has visible dynamics; and the proxy diagnostics identify both promising and weak components. The next step is to complete the policy-learning layer on top of this state representation and evaluate it with strict walk-forward backtests.

References

- CodeTrading. Reinforcement learning trading bot in python: Train an ai agent on forex (eurusd). YouTube video, 2025. URL <https://youtu.be/oW4hgB1vIoY?si=P41Wr1Kh4hIKHPBV>. Accessed: 2026-10-01.
- GeeksforGeeks. What is reinforcement learning? Web article, 2026. URL <https://www.geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/>. Updated 30 April 2026; accessed: 2026-10-01.
- Ben Hambly, Renyuan Xu, and Huining Yang. Recent advances in reinforcement learning in finance. *arXiv preprint arXiv:2112.04553*, 2023. URL <https://arxiv.org/abs/2112.04553>.
- Zhengyao Jiang, Dixing Xu, and Jinjun Liang. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*, 2017. URL <https://arxiv.org/abs/1706.10059>.
- Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance. *arXiv preprint arXiv:2011.09607*, 2020. URL <https://arxiv.org/abs/2011.09607>.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- neurotrader. How i develop trading strategies: Permutation tests and trading strategy development with python. YouTube video, 2025. URL <https://youtu.be/NLBXgSmRBgU?si=xFInVV7xdQa8hL7n>. Accessed: 2026-10-01.
- Nikolaos Pippas, Elliot A. Ludvig, and Cagatay Turkay. The evolution of reinforcement learning in quantitative finance: A survey. *ACM Computing Surveys*, 2025. doi: 10.1145/3733714. URL <https://arxiv.org/abs/2408.10932>.

- PyData. Reinforcement learning for finance. YouTube video, 2025. URL https://youtu.be/_n4JOMnAeCs?si=A5n0bqzyGdDUhsAk. Accessed: 2026-10-01.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Alberto Suarez. Reinforcement learning. Course slides, Universidad Autonoma de Madrid, 2025. Lecture material cited from the author’s course notes.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018. URL <http://incompleteideas.net/book/the-book-2nd.html>.
- zhonghong9998. Using reinforcement learning to optimize stock trading strategies. Medium article, 2024. URL <https://medium.com/@zhonghong9998/using-reinforcement-learning-to-optimize-stock-trading-strategies-a77d35ea3308>. Accessed: 2026-10-01.